

AI Architecture Decision Tool

An intelligent document-aware recommender for choosing between
RAG, Fine-Tuning, CAG and Hybrid LLM architectures

Technical Deep Dive Document

Tech stack, feature-by-feature first-principles walkthrough,
trade-offs, and depth-questions for mentors

PROJECT AT A GLANCE

Type	Full-stack AI decision support system
Stack	Next.js 16 + React 19 + FastAPI + PostgreSQL + Redis + FAISS + OpenAI
Core idea	Upload a requirements doc -> extract 10 signals -> score 4 architectures -> explain
Output	Recommendation + confidence + sensitivity + cost + PDF export
Deploy	Local dev (uvicorn + next dev), Firebase auth, containerizable

Table of Contents

1. Executive Summary
2. The Problem This Project Solves
3. Technical Stack: What + Why
 - 3.1 Frontend Stack
 - 3.2 Backend Stack
 - 3.3 AI / ML Layer
 - 3.4 Data Layer
 - 3.5 Auth and Observability
4. System Architecture Overview
5. Feature Deep Dives (First Principles)
 - 5.1 Document Upload and Parsing
 - 5.2 Text Chunking and Embeddings
 - 5.3 FAISS Vector Retrieval
 - 5.4 Signal Extraction (Hybrid LLM + Keyword)
 - 5.5 Source Verification and Anti-Hallucination
 - 5.6 Deterministic Scoring Engine
 - 5.7 Sensitivity Analysis
 - 5.8 Why-Not Explanations
 - 5.9 Follow-up Questionnaire
 - 5.10 Decision Trace Pipeline
 - 5.11 Cost Analysis Engine
 - 5.12 Results Dashboard
 - 5.13 PDF Report Generation
 - 5.14 Redis Caching Layer
 - 5.15 PostgreSQL Persistence
 - 5.16 Firebase Auth + Project Workspaces
6. Data Model
7. End-to-End Request Lifecycle
8. Trade-offs and Design Decisions
9. Future Enhancements
10. Mentor Doubts (Depth Signalling)

1. Executive Summary

The AI Architecture Decision Tool is a full-stack web application that answers a deceptively hard question every AI team eventually asks: 'For my use case, should I use RAG, Fine-Tuning, Context-Augmented Generation (CAG), or a Hybrid of these?' Instead of relying on gut feel or blog-post templates, the tool ingests a user's requirements document (PDF, DOCX or TXT), automatically extracts ten critical decision signals (like dataset size, latency budget, data volatility and accuracy needs), scores each candidate architecture using a transparent rule-based engine, validates that the recommendation is stable under input perturbation, attaches an industry-benchmarked cost estimate, and exports the whole analysis as a professional PDF report.

The system is intentionally built as a hybrid between classical information-retrieval (keyword matching + FAISS vector search), a Large Language Model (OpenAI GPT) for semantic extraction, and a deterministic symbolic scoring layer. This design is deliberate: the LLM is powerful but hallucinates, deterministic rules are trustworthy but rigid, so we sandwich the LLM between a keyword pre-pass and a verification pass, and never let it make the final decision - that's what the scoring engine is for. The end result is an explainable, reproducible recommendation with verifiable citations back to the source document.

2. The Problem This Project Solves

Building an LLM-powered product forces teams into an architectural fork in the road almost immediately. Each path has dramatically different cost, latency, maintainability and accuracy characteristics:

- RAG (Retrieval-Augmented Generation): the LLM queries a vector database at inference time. Great for volatile data but adds retrieval latency and retrieval-quality risk.
- Fine-Tuning: you retrain the model on your data. Great for specialised domains and low-latency inference, but expensive and goes stale.
- CAG (Context-Augmented Generation): stuff everything relevant into the prompt directly. Dead simple and cheap for small corpora, breaks at scale because of context-window limits.
- Hybrid: mix RAG with Fine-Tuning. Maximum flexibility but also maximum complexity and cost.

Developers usually make this choice based on whichever blog they read last. That's fine when you're wrong about a side-project, but a bad call here at an enterprise can cost hundreds of thousands of dollars and months of rework. The goal of this tool is to turn a 2-hour argument in a meeting room into a 60-second, data-driven, document-grounded, explainable recommendation that a senior engineer can defend in front of a steering committee.

3. Technical Stack: What + Why

Every component below was chosen for a specific reason. 'Use the shiniest tool' is not a reason - each pick is justified against the constraints of the problem: latency, honesty (no hallucinations), explainability, developer velocity and deploy-ability.

3.1 Frontend Stack

Next.js 16 (App Router)

React meta-framework with server components, file-system routing and streaming. Chosen because architecture-decision reports are content-heavy and benefit from SSR-friendly pages, SEO, and the App Router's nested layouts for the project workspace / analysis detail nested routes. The router also makes the `/projects/[projectId]/analyze/[sessionId]` and `/results/[analysisId]` dynamic routes trivial.

React 19

Latest concurrent React with `useTransition` and `Suspense`. Picked because our analysis

TypeScript

dashboard streams in signals, scores and cost data progressively, and concurrent rendering keeps the UI responsive while the 10+ animated charts mount.

Our API responses have deeply nested shapes (Signals, Scores, CostAnalysisData, SensitivityResult). TypeScript catches shape mismatches at build time instead of at 2am in a demo.

Tailwind CSS v4

Utility-first CSS. Chosen because the UI is many small composable panels (glass-panel, trace steps, signal cards) and Tailwind keeps the components self-contained without CSS files fighting each other.

Framer Motion 12

Declarative animation library. Used for the hero scroll-transforms, the decision-trace step animations, and page transitions. We need animation that respects motion-preferences and plays well with React concurrent mode - Framer handles that.

Lenis

Smooth inertial scrolling for the marketing landing page. Purely an aesthetic choice, but it's what makes the app feel premium rather than 'built by an engineer.'

Recharts

React chart library built on D3. Chosen over raw D3 because the four architecture-score bars, cost breakdown charts and sensitivity plots only need standard chart types - Recharts gives us those declaratively in ~10 lines each.

Three.js + Vanta

Animated 3D backgrounds on the hero section. Three.js is the WebGL standard and Vanta is a thin wrapper that gives us a 'wow' visual without writing shaders ourselves.

react-dropzone

Drag-and-drop file upload. We considered rolling our own, but dropzone handles file-type gating, drag-preview state, and keyboard accessibility correctly.

Firebase JS SDK

Google auth + session management on the client. See section 5.16.

lucide-react

Icon set. Chosen because it's tree-shakable (only the icons you import ship in the bundle) and design-consistent.

3.2 Backend Stack

FastAPI 0.115

Async Python web framework. Chosen over Flask/Django because (a) it's fully async which matters when we're doing parallel OpenAI calls and DB writes, (b) auto-generates /docs (Swagger) from Pydantic models which we use for debugging, and (c) Pydantic v2 gives us free request validation.

Uvicorn (standard)

ASGI server. The 'standard' extra pulls in uvloop on Linux and httptools for faster request parsing. Running with --reload in dev and workers>1 in prod.

Pydantic v2

Data validation. Every request body and response model is a Pydantic class - typed, auto-validated and auto-documented. We use JSONB columns on the DB side and Pydantic coerces them to typed Python dicts on the way out.

SQLAlchemy 2.0 (async-capable)

ORM. We use the sync engine for simplicity (our queries are short) but the codebase is structured so we can switch to async sessions without touching business logic. The Mapped[] / mapped_column() 2.0-style models give us type-checked relationships.

Alembic

DB migrations. Schema changes are checked into git as versioned migration files instead of hand-edited CREATE TABLE statements - this is what lets multiple developers evolve the schema safely.

psycopg2-binary + asyncpg

PostgreSQL drivers. psycopg2 for sync migration scripts, asyncpg available for async session mode.

python-multipart

Required by FastAPI to parse multipart/form-data uploads. Without it /upload would

	return 500.
python-dotenv	Loads the backend .env file. We never commit secrets - they live in .env and are read at startup.
python-jose	JWT handling. Used to verify the Firebase ID tokens on the backend so we can trust the user_id that the frontend claims.
aiofiles	Non-blocking file I/O. When we save uploaded documents to /tmp we do it via aiofiles so the event loop is not blocked during disk writes.

3.3 AI / ML Layer

OpenAI (GPT-4o mini / GPT-4)	Primary LLM for signal extraction. Chosen because (a) GPT has the strongest JSON-mode adherence, (b) the 8k prompt truncation we do still captures enough context for extraction, (c) we get function-calling-style JSON reliably. We use temperature=0.1 for extraction so outputs are near-deterministic.
Ollama (pluggable fallback)	Local LLM runtime. The LLMClient abstracts provider so you can run llama3 locally if OpenAI is unavailable or if the user needs on-prem. This was a deliberate abstraction - we never hard-coded OpenAI inside the extractor.
tiktoken	OpenAI's tokenizer (cl100k_base). We use it in the chunking utility to count tokens accurately rather than guessing with character counts, so our chunks reliably fit the embedding model's input window.
text-embedding-3-small (OpenAI)	Embedding model. 1536 dimensions, very cheap per 1k tokens, strong retrieval quality. Chosen over local models because batching via the OpenAI API is fast and we don't want to ship GPU infra for embeddings.
FAISS (faiss-cpu)	Facebook AI Similarity Search. In-process vector index. We use IndexFlatL2 with L2-normalized vectors which gives us exact cosine similarity. Chosen over Pinecone/Weaviate because for a per-session index of a few hundred chunks a local index is instant, zero-cost, zero-network-roundtrip, and persistable as a file on disk.
scikit-learn	Used for a few utility statistics (cosine similarity, normalisation) outside the FAISS hot path.
NumPy 1.26	Vector math substrate for FAISS input and embedding arrays. Pinned to <2 because FAISS ships wheels compiled against 1.x.
PyMuPDF (fitz)	PDF text extraction with page-level granularity. This is crucial: we need to know which page a quoted signal came from so that when the user reads the report they can jump back to the source. PyMuPDF is the fastest pure-Python PDF parser we benchmarked.
python-docx	DOCX parser. Same role as PyMuPDF for Word documents. DOCX is treated as a single page since the format has no native pagination.

3.4 Data Layer

PostgreSQL 14+	Source of truth. We use JSONB columns heavily for ranking/scores/decision_breakdown/why_not/followup_questions - these are flexible-shape objects that would be painful to normalise but are perfectly fine as JSON documents inside a relational row. This gives us the benefits of a document store (flexible schemas for analysis results) plus the benefits of a relational DB (FK integrity, ACID, transactions).
-----------------------	--

Redis 5 (or Upstash)	Low-latency cache. Analysis results are expensive to recompute (OpenAI calls + FAISS + scoring + sensitivity). Once computed they're keyed by session_id and cached in Redis for 15 minutes. First read is slow, subsequent reads are ~1ms. See section 5.14.
RQ (Redis Queue)	Pinned in requirements but the current hot path is synchronous. RQ is staged for future async background jobs (see section 9).
FAISS on-disk sidecar	Each session gets its own directory under faiss_index/<session_id>/ containing index.faiss and meta.json. We don't share an index across sessions because (a) session documents are isolated and (b) it lets us blow away old sessions cleanly with rmtree.

3.5 Auth and Observability

Firebase Authentication	Google Sign-In. Chosen because a) we don't want to implement password hashing, reset flows, email verification - Firebase gives us all of that for free, b) the Firebase ID token can be sent to the backend and verified, and c) Firebase's JS SDK hooks into React seamlessly.
Python logging module	Structured logs. uvicorn/sqlalchemy loggers are dialled down to WARNING so our app's INFO logs stand out during debugging.
fpdf2 (on server side)	PDF generation. Used both for the cost-analysis PDF and the main recommendation PDF. Chosen over ReportLab because fpdf2 is simpler, ships fewer native deps, and is easy to theme with our brand colours. See section 5.13.

4. System Architecture Overview

The system follows a classic 3-tier layout with an additional AI-processing lane:

- Presentation: Next.js 16 app, runs in the browser, talks to the backend over HTTP/JSON. Routes: / (landing), /projects (workspace), /projects/[id]/analyze (upload flow), /results/[analysisId] (dashboard).
- API: FastAPI mounted at /api/v1 with five routers - upload, analysis, questionnaire, projects, users. All endpoints are async and return Pydantic models.
- Services layer: pure-Python modules under backend/services (document_parser, signal_extractor, scoring_engine, followup_generator, llm_client) plus backend/app/services (signal_service, recommendation_service, vector_service, cost_analysis, pdf_report, cache_service). This split is intentional: services/ are domain logic with no framework dependency, app/services/ are orchestration that uses the DB session.
- Data: PostgreSQL for structured state (users, projects, sessions, signals, results), Redis for cache, FAISS on the local filesystem for vector indices.
- External: OpenAI for LLM + embeddings, Firebase for auth.

5. Feature Deep Dives (First Principles)

5.1 Document Upload and Parsing

What it does

You drag a PDF, DOCX or TXT file into the upload zone on the frontend. A few seconds later, you have a structured Python dict back that contains the full text, the text broken up by page number, word and character counts, and the filename. This dict is the raw material everything downstream consumes.

How it works, step by step

- Frontend: react-dropzone component captures the file, validates the MIME type against a whitelist, and POSTs a multipart/form-data request to `/api/v1/upload`. A progress indicator is shown as soon as `onDrop` fires.
- Backend: FastAPI's `UploadFile` reads the bytes into memory. We first call `DocumentParser.validate_file()` which checks extension (`.pdf/.docx/.txt` only) and size (`<=50 MB`). If either fails we raise `HTTPException(400)` and the client surfaces the error.
- We save the bytes to a temp file via `tempfile.mkdtemp()` rather than keeping them in RAM, because `PyMuPDF` and `python-docx` expect file paths. The temp dir is cleaned up in the finally block regardless of success or failure.
- The parser dispatches on extension: `.pdf` -> `PyMuPDF` iterates `doc[i].get_text('text')` for each page, preserving `page_number`. `.docx` -> `python-docx` extracts paragraphs and joins them (DOCX has no real page concept so we expose it as a single page). `.txt` -> plain read with `errors='replace'` for robustness against bad encoding.
- The output is a dict with `full_text`, `pages[]`, `total_pages`, `char_count` and `word_count`. `full_text` is the joined per-page text with `'\n\n'` separators - this boundary marker helps later when we try to map a quoted snippet back to its source page.

Why PyMuPDF specifically

PDF text extraction is deceptively hard: layout analysis, font encoding, ligatures, figures, 2-column layouts. We benchmarked `PyPDF2`, `pdfminer.six` and `PyMuPDF`. `PyMuPDF` (fitz) wins on speed (pure C under the hood) and layout accuracy, and importantly it returns text per page natively which is essential for source-citation.

Trade-offs

- We don't OCR scanned PDFs. If the PDF is a scanned image, `PyMuPDF` returns empty strings and we'll raise a 422 'too little text'. Adding `Tesseract/PaddleOCR` would fix that but would triple the container size.
- DOCX is single-page. If a user uploads a 200-page Word doc we lose per-page attribution. Acceptable because most real uploads are PDFs.
- 50 MB cap. Larger files are probably corpora, not requirements docs, and would blow our embedding budget anyway.

5.2 Text Chunking and Embeddings

What it does

Long documents can't be embedded in one shot - embedding models have a max input token limit (roughly 8k tokens for `text-embedding-3-small`). Chunking slices the document into overlapping windows that each fit, and embedding turns each chunk into a 1536-dim vector that encodes its meaning.

The chunking algorithm (backend/app/utils/embeddings.py)

- We tokenize with `tiktoken` using `cl100k_base` - the exact tokenizer the embedding model uses. Counting tokens exactly (not chars, not words) guarantees our chunks will fit.
- We slide a window of `CHUNK_SIZE` tokens across the token stream, stepping by `(CHUNK_SIZE - OVERLAP)`. Typical values: `chunk_size=400`, `overlap=50`. The overlap matters: without it, a sentence sitting on the boundary would be split and neither chunk would contain it wholly, hurting retrieval.
- Each chunk is decoded back to text with `enc.decode()` so we can store human-readable strings in the FAISS meta sidecar.

Embedding

- `embed_texts()` batches in groups of 100 (OpenAI limit) and does async HTTP via `AsyncOpenAI`. Each returned embedding is converted to a float32 numpy array - float32 (not float64) because FAISS requires float32 and because we halve memory for free.
- Fallback: if `OPENAI_API_KEY` is missing, we return zero vectors. This is a graceful degradation path - retrieval will be useless but the app won't crash during local dev without an API key.

Why overlapping chunks and why L2-normalise later

Overlap protects against boundary effects - a query might match content that straddles two chunks, and overlap guarantees at least one chunk contains both halves. L2-normalising the vectors before indexing (which we do inside `faiss_store.add_embeddings`) lets us use `IndexFlatL2` as if it were cosine similarity: for unit vectors, L2 distance and cosine distance are monotonically equivalent.

Trade-offs

- Chunk-size is a classic precision/recall knob. Smaller chunks = more precise matches but more chunks to process; larger chunks = more context per match but sloppier retrieval. 400 tokens is a sweet spot for requirement documents.
- Using OpenAI embeddings costs money and phones home - a privacy-conscious deployment could swap in sentence-transformers locally, which is why `LLMClient` is pluggable even though `embed_texts()` currently hard-codes OpenAI.

5.3 FAISS Vector Retrieval

What it does

FAISS is the session-scoped semantic memory of an uploaded document. When we need to feed the most relevant chunks of the document into the LLM for signal extraction, FAISS gives us those top-k chunks in microseconds.

Why per-session indices

Every upload creates a new directory: `faiss_index/<session_id>/index.faiss` and `meta.json`. We explicitly do NOT share an index across sessions because: (a) cross-session contamination would be a privacy and correctness disaster, (b) it makes cleanup trivial (`rmtree` on session delete), (c) it lets us pick a dumb-but-perfect index type - `IndexFlatL2`, which does exhaustive L2 search - since each index only has a few hundred vectors where sublinear methods give you nothing.

Search logic

- We embed the query (in our pipeline the query is 'architecture requirements dataset latency' - a hand-crafted anchor that pulls the most signal-relevant chunks). We L2-normalise it, reshape to (1, dim), and call `index.search(vec, top_k)`.
- The raw FAISS result is a list of integer indices. We look those indices up in the `meta.json` sidecar to get back the text chunk and page number. That's what we feed to the LLM.

The sidecar meta file

FAISS only stores vectors - not the original text. We keep `meta.json` alongside the index: a list of {`chunk_id`, `session_id`, `text`, `page`}. On `add_embeddings()` we append new entries; on search we look up by row index. This is simpler than FAISS's own id-mapping and works because we only ever append, never delete individual vectors.

Trade-offs

- `IndexFlatL2` is $O(n)$ per query. Fine for $n < 10k$. If we ever index whole corpora instead of single docs we'd need IVF/HNSW.
- Per-session indices mean we can't ever do cross-document retrieval (e.g. 'find previous projects that looked like this one'). That's a future enhancement - see section 9.

5.4 Signal Extraction (Hybrid LLM + Keyword)

The ten signals

We extract exactly ten signals from the document. They were chosen because these are the ten knobs that actually differentiate RAG vs FineTuning vs CAG vs Hybrid in real projects:

- dataset_size: small / medium / large / very_large
- query_volume: low / medium / high / very_high
- latency_requirement: relaxed / moderate / strict / ultra_low
- data_volatility: static / low / moderate / high
- accuracy_requirement: moderate / high / very_high / critical
- domain_specificity: general / moderate / specialized / highly_specialized
- security_level: standard / elevated / high / critical
- cost_sensitivity: low / moderate / high / very_high
- deployment_preference: cloud / on_premise / hybrid / edge
- user_scale: small / medium / large / enterprise

Three-pass extraction pipeline

We do not just 'ask the LLM'. That would be fast and wrong. Instead we run three passes and merge them:

- Pass 1 - Keyword scan: for each signal, we check the document text for a list of domain keywords (e.g. dataset_size -> 'corpus', 'records', 'terabyte'). If any hit, we extract the surrounding sentence and record a low-confidence match. Keywords alone can't tell you if the dataset is 'large' or 'small', so the value is left null - this pass only proves 'the document talks about this topic'.
- Pass 2 - LLM extraction: we send the (optionally FAISS-retrieved) document text to GPT with a strict prompt that tells it (a) only output JSON, (b) never hallucinate, (c) copy the supporting quote verbatim, (d) include a page number. Temperature 0.0 for maximum determinism.
- Pass 3 - Source verification: we check that every verbatim quote the LLM returned actually appears in the document. If the exact substring is missing we fall back to a fuzzy-prefix search (take the first N words of the claimed quote and see if any N>=60% of them appear consecutively). If neither succeeds, we cut the signal's confidence in half and flag it source_verified=false.
- Merge: we combine keyword confidence and LLM confidence ($\text{llm_conf} + 0.3 \cdot \text{kw_conf}$ clamped to 1). The LLM's value wins because only the LLM can discriminate among the bucket options. If combined confidence falls below 0.1 we null the value entirely - hallucinated signals should not propagate.

Why this multi-pass design

The LLM is the only part that can turn 'we get about 50 million records a day' into 'very_large'. The keyword pass is there to catch cases where the LLM misses a topic the document clearly discusses, boosting combined confidence. The verification pass is the anti-hallucination guardrail - GPT is capable of fabricating quotes that sound real. By matching quotes back to the source text we catch fabrication in milliseconds. This is the difference between a tool you can defend in a meeting and a toy demo.

Trade-offs

- We truncate the document to the first 8000 characters before sending to the LLM. Cheap, deterministic, and usually the requirements doc has the punchy summary early. Larger docs are handled via the FAISS pre-retrieval that prepends the most semantically relevant chunks before truncation.
- Keyword lists are hand-curated per signal. They are a maintenance burden but they make the extractor robust against LLM drift.

5.5 Source Verification and Anti-Hallucination

The core trick

The LLM outputs a 'source_text' field with what it claims is a verbatim quote. We then do a case-insensitive substring search for that quote in the full document. If we find it, source_verified=true and we even correct the page number by searching per-page text. If we don't find it, we don't trust the LLM - we try fuzzy recovery first (maybe GPT slightly reworded), and if even that fails we halve the confidence and mark source_verified=false.

The fuzzy recovery

We tokenise the claimed quote into words, and progressively shorten the prefix. 'We process 10 million records per day' might become 'We process 10 million records' if the original document said 'We process 10 million records daily'. If any 60%-length-or-more prefix is found, we expand out to sentence boundaries (back to the previous period and forward to the next) and use that as the authoritative source. This lets us recover from minor LLM paraphrasing without swallowing fabrications.

Why this matters to users

In the final report, every signal carries a `source_text`, `page_number` and `source_verified` flag. The frontend renders verified signals in green with a citation, unverified in amber. A developer reviewing the report can always click through to the exact sentence that drove the recommendation - no black box.

5.6 Deterministic Scoring Engine

Why rules-based and not 'ask the LLM'

The scoring engine is intentionally boring. Given ten signals, it computes four architecture scores using a hand-tuned lookup table. No LLM, no randomness, no temperature. Two reasons: reproducibility (the same inputs must always produce the same recommendation) and defensibility (we can point at a specific rule and explain why RAG outscored FineTuning).

The data model

- **SCORING_RULES:** a three-level dict. `signal` -> `signal_value` -> `architecture` -> `score` [0.0 to 1.0]. Example: `data_volatility='high'` gives `RAG=1.0` (retrieval shines with live data) and `FineTuning=0.2` (fine-tuned models go stale fast).
- **SIGNAL_WEIGHTS:** each signal has a weight. `data_volatility=1.3`, `accuracy=1.2`, `deployment_preference=0.7`. Weights were chosen by thinking through which signals actually should dominate the decision - volatility is a hard constraint, deployment pref is softer.

The math

For each signal, we compute `effective_weight = signal_weight * confidence`. If the LLM said 'high' with confidence 0.9 for `data_volatility`, the effective weight is $1.3 * 0.9 = 1.17$. We then add `signal_score[arch] * effective_weight` to each architecture's running total, and accumulate `total_weight`. At the end we normalise: `final_score[arch] = total[arch] / total_weight * 100`. Missing or very-low-confidence signals are simply skipped - they contribute nothing rather than pushing the answer toward an unfounded default.

Suitability buckets

After normalising, each architecture is labelled: `>=75` 'Highly Suitable', `>=55` 'Suitable', `>=40` 'Moderately Suitable', else 'Not Recommended'. The frontend colours bars accordingly. These thresholds are tunable and are the same for all sessions.

Overall confidence

`confidence = 0.6 * avg_signal_confidence + 0.4 * coverage`, where coverage is the fraction of signals above 0.3 confidence. Two components because you can have high individual confidences but only 3/10 signals - that's low coverage - or you can have 10/10 signals but each only 0.4 confidence - that's low individual trust. We want both.

Trade-offs

- Hand-tuned weights are a form of expert knowledge baked into code. Easy to audit, but you need a human to change them. An ML alternative would be to learn weights from labelled past projects - great when you have those labels, which at project start you don't.
- The engine only scores four architectures. Adding a fifth (e.g. agentic workflows) means adding a column to every row of **SCORING_RULES**.

5.7 Sensitivity Analysis

What it is

A recommendation is only useful if it's stable. If flipping `data_volatility` from 'moderate' to 'high' changes the winner, the user should know. Sensitivity analysis is the stress-test: perturb each signal's value to each other legal value, re-run the scorer, and count how many perturbations change the recommended architecture.

Algorithm

- Start from the base recommendation computed on the real signals.
- For each of the 10 signals, for each of the 3 alternative values, swap it in (clamping confidence to at least 0.5 so the change has real weight) and call `score()` on the perturbed signal dict.
- If the new recommended architecture differs from the base, record (`signal`, `original_value`, `perturbed_value`, `new_rec`, `score_delta`).
- Compute `stability = 1 - (instability_count / (num_signals * 3))`. Stable is >0.7 .

Why this matters

Instead of saying 'Use RAG', we say 'Use RAG; this is stable under 28 of 30 perturbations; it would flip to Hybrid only if `data_volatility` changed to static'. That's the kind of answer a senior engineer trusts.

Trade-offs

- Sensitivity runs 30 extra `score()` calls per analysis. The scoring function is pure Python dict lookups so it's ~1ms - negligible.
- We don't perturb pairs of signals simultaneously. That would give us a more rigorous stability score ($O(n^2)$) instead of $O(n)$ but 30 single-variable perturbations catch the fragility cases in practice.

5.8 Why-Not Explanations

For every non-recommended architecture, we produce a short sentence explaining why. We take the gap in score between the recommendation and the alternative, and list the three weakest factors that alternative scored on. Example: 'FineTuning scored 18.3 points lower than RAG. Weakest on: data volatility, dataset size, deployment preference.' This is purely mechanical (no LLM) so it's reproducible and fast.

5.9 Follow-up Questionnaire

What triggers it

After extraction, any signal with `value=null` or `confidence<0.3` is 'missing'. We sort missing signals by priority (`dataset_size` and `data_volatility` are priority 5, `user_scale` and `deployment` are priority 2) and generate up to 5 follow-up questions with pre-defined multiple-choice options.

Why MCQ

Free-text answers would need re-parsing through the LLM, which is where we started. Multiple-choice answers map 1:1 to signal values - no ambiguity, no cost, no extra LLM call.

How answers flow back

- User picks answers in the UI, frontend POSTs `/api/v1/followup` with `{analysis_id, answers: {signal_name: value}}`.
- Backend updates the Signal rows for the affected signals (`confidence=1.0`, `source='Follow-up answer from user'`, `source_verified=True`).
- It invalidates the Redis cache for that session and calls `recommendation_service.score_and_persist()` again - new rows in Result, same `session_id`.
- Response returns the new full `AnalysisResponse` including updated scores and updated followups list.

Trade-offs

- Max 5 questions because asking more is user-hostile. If the document truly has <5 signals extracted, we prioritise the most impactful ones.
- The answer set is fixed to the same bucket labels as extraction, so the scoring engine doesn't need to know whether a signal came from extraction or from a questionnaire.

5.10 Decision Trace Pipeline

A `decision_trace` is a list of steps, each with `{step, status, timestamp, details}`, recorded by the upload router as processing progresses: `upload -> parse -> section_detection -> vector_indexing -> signal_extraction -> missing_signals -> scoring -> recommend`. The frontend `DecisionTrace` component renders these as an animated vertical timeline. This is how the user sees that signal extraction is actually running - not a spinner. It's also invaluable for debugging: if a session errors out, the trace tells us exactly which stage failed.

5.11 Cost Analysis Engine

What it computes

Given the extracted signals plus the recommended architecture, we estimate monthly and annual cost per architecture across seven categories: compute, storage, API/inference, networking, training, maintenance, and a security premium. Output also includes setup cost, cost-per-query, and a 'best value' ranking.

The cost model

- Each category has a nested dict: `signal -> value -> architecture -> [low, high]` range. These ranges are benchmarked from public cloud pricing (AWS, Azure, GCP) and typical fine-tuning budgets for GPT-scale models.
- Deployment multiplier: `on_premise = 1.4x` maintenance, `edge = 1.5x`, `cloud = 1.0`. This captures the harsh reality that self-hosting is cheaper on paper but expensive in ops.
- Security premium: `critical = 1.5x` overall. Compliance work (HIPAA, GDPR, SOC2) costs real money and this captures it.
- Monthly total = `compute + storage + inference + networking + training + maintenance + security premium`, each expressed as a range.
- Annual total = `monthly_total * 12 + setup_cost`.
- Cost per query = `monthly_total / monthly_query_volume`, derived from the `query_volume` bucket.

Best-value ranking

`efficiency[arch] = suitability_score / (avg_monthly_cost / 1000)`. This normalises 'score per \$1k spent per month'. The architecture with the highest efficiency is the 'best value' - it might be different from the recommended one if the recommendation is Ferrari-class and something cheaper would do.

Why hard-coded ranges and not live pricing

Live pricing APIs (AWS Price List API) exist, but they return raw SKUs, not 'what would this architecture cost'. Translating SKUs into an end-to-end cost model is a multi-week project in itself. The hand-curated ranges are good enough for a decision-making tool and are honest about being a range (low-high), not a point estimate.

Trade-offs

- Numbers drift as cloud pricing changes. Cost-model ranges should be refreshed every ~6 months.
- We don't model cold-start vs steady-state costs separately.

5.12 Results Dashboard

The `/results/[analysisId]` page stitches together every component the backend produced: ranked architecture bar chart, confidence gauge, signal cards with verified-source badges, why-not callouts, sensitivity warning banner, cost-analysis

accordion, decision-trace timeline, and follow-up questionnaire modal. Data is fetched once via `getAnalysis(id)` and rendered from a single `AnalysisResult` object typed in `lib/api.ts`. Framer Motion staggers the mount animations so the page 'builds up' visually instead of popping in.

5.13 PDF Report Generation

What it does

Two PDF exports: main recommendation report (`GET /api/v1/export/{id}`) and cost-only report (`GET /api/v1/export/{id}/cost`). Both use `fpdf2` with a custom subclass (`_ReportPDF`) that overrides `header()` and `footer()` to draw a branded blue bar and page-number line.

Why fpdf2

We considered `ReportLab` (more powerful but heavier API) and `wkhtmltopdf` (render HTML -> PDF, but requires a native binary). `fpdf2` is pure Python, small, and our reports are structured enough (headings, tables, callouts) that we don't need full HTML rendering. The custom helpers (`_section`, `_safe`, suitability colour mapping) give us brand consistency.

Latin-1 sanitisation

Out of the box `fpdf2` uses latin-1 encoding for built-in fonts. Documents we parse often contain curly quotes, en-dashes and bullet characters. `_s()` and `_safe()` normalise those to latin-1-safe ASCII equivalents so the PDF never crashes on an exotic character.

5.14 Redis Caching Layer

Two namespaces

signals:<session_id> and result:<session_id>. TTLs: signals 10 min, result 15 min. Every expensive read hits cache first, misses fall through to Postgres, and we write-through after recompute. The `cache_service` module is the only place that knows about Redis - all the business logic just calls `get_signals()/set_signals()`. If Redis is down, `_client` is `None` and every call is a no-op - the app silently degrades to 'slower but correct'.

Why not cache everything forever

15 minutes is enough to survive a page refresh or a user sharing a URL with a teammate in the same meeting. Longer TTLs would mean stale results if the user submits follow-up answers and we missed invalidation. We actively invalidate on followup via `cache_service.delete('signals', session_id)` to force a fresh read.

Upstash compatibility

The `cache_service` accepts `REDIS_URL` + `REDIS_TOKEN` so the same code works against local Redis (`redis://localhost:6379`) and managed Upstash (`rediss://...`). The SSL is inferred from the URL scheme. One less env-flag.

5.15 PostgreSQL Persistence

Tables

- users: id=Firebase UID, name, email, provider, photo_url, created_at, updated_at.
- projects: uuid pk, user_id (nullable string FK to users), name, description, status, analysis_id, mode, timestamps.
- sessions: uuid pk, project_id FK, status enum(draft/processing/completed/error), provider, filename, timestamps.
- signals: uuid pk, session_id FK CASCADE, signal_name, value, confidence, source_text, page_number, source_verified.
- results: uuid pk, session_id FK unique CASCADE, recommended_architecture, confidence_score, ranking(JSONB), scores(JSONB), decision_breakdown(JSONB), why_not(JSONB), suitability(JSONB), followup_questions(JSONB),

sensitivity(JSONB), decision_trace(JSONB), architecture_details(JSONB), created_at.

Why JSONB for results but not for signals

Signals are queryable - we sometimes want to filter 'how many sessions had data_volatility=high?' - so they're normalised into one row per signal. Results are blobs we read back whole - no queries ever say 'find sessions whose scores.RAG > 80' so they're fine as JSONB. JSONB is fast to read and supports GIN indexes if we ever need them.

Cascades

On session delete, signals and result CASCADE delete. Projects FK on session is SET NULL - if a project is deleted, sessions become orphaned but still readable for history.

5.16 Firebase Auth + Project Workspaces

Firebase handles Google sign-in, tokens, refresh and session persistence entirely on the client. Our frontend stores the uid in auth-context.tsx (React context) and passes it implicitly via fetch headers. The backend uses the uid as the users.id primary key. Projects belong to users via user_id. The projects-store.ts is a small client-side store (based on React hooks + localStorage) that caches project metadata offline so the workspace page loads instantly even before the backend responds. When a project is created via CreateProjectModal, we POST to /api/v1/projects and get back a UUID that we then use to scope subsequent /upload calls (via the optional project_id query param). This keeps all analyses for a single project grouped together in the UI.

6. Data Model (Quick Reference)

User 1-N Project 1-N Session 1-N Signal; Session 1-1 Result. All UUID primary keys (except User which uses Firebase UID). Timezones stored as TIMESTAMP WITH TIME ZONE everywhere - the db/models.py now_utc() helper guarantees inserts are UTC-aware.

7. End-to-End Request Lifecycle

A full request traced from click to dashboard:

- 1. User logs in via Firebase Google popup. Firebase returns an ID token; frontend stores user in auth-context.
- 2. User clicks 'New Project', fills the modal, frontend POSTs /api/v1/projects. Backend inserts into projects table and returns UUID.
- 3. User drags a PDF into DocumentUpload. Frontend builds FormData and POSTs /api/v1/upload?provider=openai&project_id=<uuid>.
- 4. Backend validates file extension+size, writes to tempfile, inserts a Session row with status=processing.
- 5. DocumentParser extracts pages[] + full_text. detect_sections runs keyword-based section heuristics.
- 6. vector_service.index_document chunks + embeds + writes to per-session FAISS index.
- 7. signal_service.extract_and_persist retrieves top FAISS chunks, prepends them to the text, calls SignalExtractor. Pass 1 keyword, Pass 2 LLM, Pass 3 source verification, merge, anti-hallucination null-out. Persisted to Signal rows, cached in Redis.
- 8. recommendation_service.score_and_persist runs ScoringEngine.score() -> scores+ranking+why_not+factor_breakdown, then sensitivity_analysis() -> stability, then generate_followup_questions() -> list. Upserts Result row, writes to Redis.
- 9. Session row flipped to 'completed'. Response returned.
- 10. Frontend receives AnalysisResponse, navigates to /results/[id], renders dashboard.
- 11. If followups exist and user answers them, POST /api/v1/followup triggers an update_signals -> re-score loop, Redis invalidated, new Result persisted.
- 12. User clicks 'Export PDF', frontend hits /api/v1/export/{id}, backend loads cached or DB result, fpdf2 renders,

binary returned with Content-Disposition: attachment.

8. Trade-offs and Design Decisions

Deterministic scorer vs LLM choice	Chose deterministic. Reproducible, auditable, zero-cost per call, defensible in reviews. Loses a little nuance but gains a lot of trust.
Per-session FAISS vs global index	Chose per-session. Privacy is enforced by construction, cleanup is trivial, and a global index is premature optimisation when each doc is already small.
OpenAI vs local LLM	Chose OpenAI by default with Ollama as a pluggable fallback. OpenAI wins on JSON reliability and latency; Ollama is there for on-prem demos.
PostgreSQL JSONB vs separate blobs	Chose JSONB. Resfiles are write-once-read-whole blobs; normalising them would just add joins with no query wins.
Synchronous upload pipeline vs RQ	Chose synchronous job. Upload -> response < 30 seconds in practice and the user expects to wait. Moving to RQ is trivial when we need to handle larger files.
fpdf2 vs ReportLab for PDF	Chose fpdf2. Simpler API, pure-Python, good enough for our layout.
Next.js App Router vs Pages Router	Chose App Router. Better nested layouts for /projects/[id]/analyze, first-class server components, closer to where the ecosystem is going.
Tailwind v4 vs CSS Modules	Chose Tailwind. Our components are small self-contained visual units - Tailwind keeps them literally self-contained.
Redis as cache vs PostgreSQL	Chose Redis. O(1) keyed access vs running a view refresh pipeline. Works equally well against Upstash for serverless deploys.
Source verification (fuzzy recognition)	Chose graceful penalty. A slightly paraphrased quote is usually still a good signal; hard-rejecting it throws away information.

9. Future Enhancements

- Background job queue: move /upload off the request path using RQ. The user gets an immediate session_id and polls /analysis/{id} until status=complete. This unblocks the web worker for very large documents and multi-doc uploads.
- Cross-session retrieval: a second FAISS index that stores embeddings of past successful analyses so new users can be shown 'projects similar to yours chose RAG'.
- Learned scoring weights: right now SIGNAL_WEIGHTS is hand-tuned. Once we have ~50 labelled post-hoc outcomes ('team X picked RAG, regretted it') we could learn weights with a simple logistic regression.
- Multi-document analysis: today one session = one doc. Real projects have 5+ specs. We'd need to merge extracted signals across docs with conflict resolution.
- Live cost API integration: plug into AWS Price List + Azure Retail Prices to refresh _INFRA_COST automatically each month.
- Agent architecture option: add a fifth candidate (agentic/tool-use) with its own rules and cost category. Would require extending SCORING_RULES, SIGNAL_OPTIONS, scoring_engine.architectures, and cost_analysis._ARCHITECTURES.
- RBAC: multi-tenant mode where organisations have seats and projects. Today user_id scoping is flat.
- Observability: OpenTelemetry traces across upload -> parse -> embed -> extract -> score so we can see latency waterfalls in a dashboard like Jaeger.
- Streaming responses: stream signals to the client via SSE as they extract instead of waiting for the full pipeline.

- Vector DB upgrade path: for corpus-level retrieval, swap IndexFlatL2 for HNSW or migrate to a hosted vector DB (Qdrant/Weaviate) without touching business logic because vector_service is a clean boundary.
- Automated regression tests: a golden-set of 20 requirement documents with known-good signal extractions, run in CI to catch LLM drift.
- Fine-grained audit log: every signal change and every re-score writes an entry to an AuditEvent table. Useful for enterprise adoption.

10. Mentor Doubts (Depth-Signalling Questions)

These are questions you can ask a mentor to demonstrate you have actually understood the system end-to-end. They are grouped by depth axis.

A. Questions about the AI layer

- How should we evaluate whether GPT's extraction is getting better or worse over time? Is a golden-set of labelled documents plus Cohen's kappa between extractions the right framework, or is there something more robust for structured extraction quality?
- Our anti-hallucination pass halves confidence on unverified quotes. Is a confidence halving principled, or should it be a full reject with a 're-ask LLM with stricter prompt' retry? What's the cost/precision tradeoff we should measure?
- We truncate to the first 8000 chars before sending to the LLM. FAISS retrieval prepends the most relevant chunks. If a signal is only mentioned on page 40 of a 100-page doc, does FAISS actually pull it reliably with our 400-token chunks and overlap of 50, or should we increase retrieval top_k specifically during extraction?
- Temperature=0.1 isn't zero. Is there a reason we're not using 0.0 everywhere for extraction, and should we pin a seed for full determinism?
- Have we considered structured output (function calling / JSON schema enforcement) instead of free-form JSON mode? Would it eliminate the JSONDecodeError fallback path entirely?
- Our embedding model is text-embedding-3-small (1536 dim). Would text-embedding-3-large (3072 dim) improve retrieval enough to justify the doubled cost and memory, or is the bottleneck actually the chunking strategy?

B. Questions about the scoring engine

- The SIGNAL_WEIGHTS are hand-tuned. What's a principled path to learning them from historical outcomes? Linear regression against labels like 'did the team ship on time', or is there something better like an ordinal logistic or a tree-based model?
- We skip signals with confidence < 0.1 instead of imputing a prior. Is skipping or imputing the more honest behaviour - i.e., does the total_weight normalisation already handle this correctly?
- Our sensitivity analysis perturbs one signal at a time. Given the exponential blow-up, is there a sampled multi-variable perturbation (LHS, Sobol indices) that would give a stronger stability claim without running 10⁴ scorer calls?
- The suitability thresholds (>=75, >=55, >=40) are global constants. Should they be percentile-based relative to the score distribution for a given session instead, so a narrow score cluster doesn't all fall into the same bucket?
- why_not explanations use the weakest three factors. Shapley-style per-signal attribution would be more rigorous - is the extra compute justified?

C. Questions about system architecture

- Upload is synchronous today. What's the right SLO - do we commit to <30s p95 - or do we move to an async queue and pay the complexity tax? At what throughput does the sync model break down?
- Per-session FAISS indices are stored on local disk. In a multi-replica deploy that breaks - replicas don't share disk. Do we move the index to object storage (S3) and load on demand, or into a managed vector DB? What are the tradeoffs?

- Redis cache TTLs are 10-15 minutes. With follow-up answers we invalidate explicitly. Is there a case for write-through with longer TTLs and explicit versioning in the cache key (e.g. result:v2:<id>) to avoid invalidation bugs?
- Our CORS is `allow_origin_regex='.*'` - fine for local dev. What's the hardening story for production?
- Sessions reference projects with `ON DELETE SET NULL`. Should this be `CASCADE` so deleting a project really nukes its history? This is a product question as much as a tech one.

D. Questions about data and security

- We store the full `source_text` for every signal (up to 2000 chars) in Postgres. If a customer uploads a document with PII, that PII is now in our DB. Do we need an ephemeral mode that keeps only hashes of source text and not the text itself?
- Firebase ID tokens are verified client-side for UX but our backend does not (yet) verify them. Is this a gap? If yes, do we use `python-jose` to verify the JWT signature against Firebase's JWKS, or do we run Firebase Admin SDK server-side?
- GDPR: if a user deletes their account, what's the complete erasure path? Today we have `SET NULL` on `projects.user_id` which would leave orphan rows - is that acceptable, or do we need a hard cascade?
- Does OpenAI retain the prompts we send? If yes, is that a data-exfiltration concern for enterprise customers and should we document a 'zero-retention endpoint' option?
- The per-session FAISS sidecar `meta.json` contains raw chunk text. If those files are never explicitly deleted (we only `rmtree` on `delete_session_index`), do we have a leak path where old sessions persist forever on disk?

E. Questions about cost analysis realism

- Our cost ranges are static dictionaries. How often should they be refreshed, and is there a way to ground them in a cloud pricing API like AWS Price List so they self-update?
- We compute cost-per-query using a single bucket (low=1000, medium=10k, etc.). Cloud bills are non-linear at the tail because of committed-use discounts and spot pricing. Should we model a utilisation curve instead?
- Our 'best value' score is `suit_score / avg_cost`. That's one flavour of efficiency. Should we also expose Pareto frontier data so users can pick their preferred trade-off explicitly?
- Do we account for egress costs? Networking only includes ingress and internal - and for LLM-heavy products egress can dominate.

F. Questions about future enhancements and direction

- If we added support for 'agentic' architectures as a fifth option, which of the ten signals would even discriminate it from Hybrid - or do we need a new signal entirely (e.g. 'tool availability')?
- Multi-document projects: how should we reconcile conflicting extracted signals across docs? Confidence-weighted vote, recency, or flag the conflict and let the user resolve it?
- Would a learned scoring model (supervised on outcomes) undermine the deterministic auditability story the project leans on? How do we get both learnability and explainability - SHAP on top of a tree model?
- Where is the product heading - is this a standalone SaaS, an internal HPE tool for solution architects, or a component we want to embed into another product? The answer changes whether we prioritise multi-tenancy or deep integration APIs.
- If a mentor were to pick ONE feature to invest the next sprint on that would move the needle most for enterprise adoption, which would they pick: hard source-verification, SSO, zero-retention mode, or RBAC?

G. Sharp, opinion-forcing questions (asked last)

- You've now read the scoring table. Is there a signal-value cell in `SCORING_RULES` that you think is flat-out wrong, and why? (This forces the mentor to engage with the rule matrix itself.)
- If we had unlimited budget for one month, would you spend it on improving extraction recall, scoring weights, or

UX/dashboard polish? Why that ordering?

- What would make you stop trusting this tool as a user? Concretely - what output would make you close the tab and go back to picking by gut?

CLOSING NOTE

This document was generated from a direct read of the live codebase: backend routers, services, scoring_engine, signal_extractor, cost_analysis, pdf_report, db models, frontend lib/api.ts, components/ and package.json. Every claim above is grounded in code you can open right now - no fluff, no filler.